

Ejercicio 2

```

1 // 4. Una línea diagonal
2 ssd1306_line(10, 50, 110, 10);
3 ssd1306_flush();
4 vTaskDelay(pdMS_TO_TICKS(500));
5 // 5. Un rectángulo hueco
6 ssd1306_rect(12, 12, 35, 15);
7 ssd1306_flush();
8 vTaskDelay(pdMS_TO_TICKS(500));
9 // 6. Un rectángulo relleno (usando
10 //    ssd1306_box)
11 ssd1306_box(95, 20, 15, 35);
12 ssd1306_flush();
13 vTaskDelay(pdMS_TO_TICKS(500));
14 // 7. Un círculo hueco
15 ssd1306_circle(64, 32, 15);
16 ssd1306_flush();
17 // 8. Un círculo relleno (usando
18 //    ssd1306_disc)
19 ssd1306_disc(30, 45, 15);
20 ssd1306_flush();
21 vTaskDelay(pdMS_TO_TICKS(500));
22 // 9. Marco alrededor de todo
23 ssd1306_rect(0, 0, 128, 64);
24 ssd1306_flush();
25 vTaskDelay(pdMS_TO_TICKS(500));

```

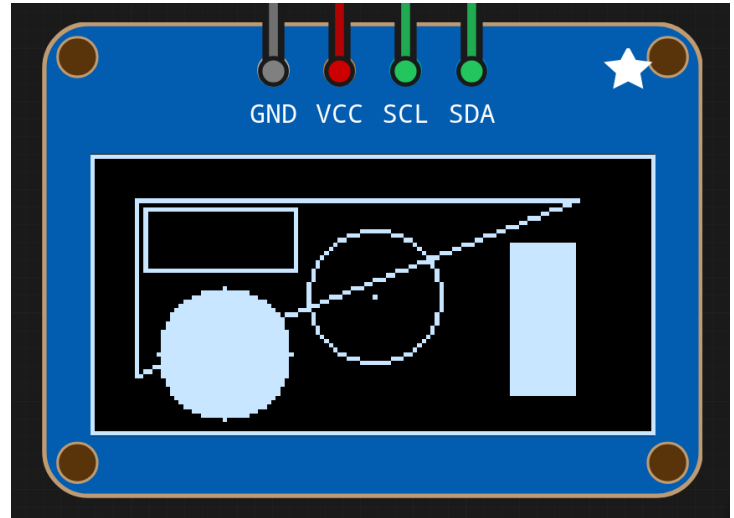


Figure 1: Secuencia completa con todas las figuras.

Ejercicio 3

```

1 #include <math.h>
2 #ifndef M_PI
3 #define M_PI 3.14159265358979323846
4 #endif
5 void app_main(void) {
6     ssd1306_init(5, 6);
7     float angulo = 0.0;
8     int centro_x = 64;
9     int centro_y = 32;
10    int radio = 31;
11    while(1)
12    {
13        ssd1306_clear();
14        int fin_x = centro_x + (int)
15            (radio * cos(angulo));
16        int fin_y = centro_y + (int)
17            (radio * sin(angulo));
18        ssd1306_line(centro_x,
19            centro_y, fin_x, fin_y);
20        ssd1306_circle(centro_x,
21            centro_y, radio);
22        ssd1306_flush();
23        angulo += 0.1;
24        if (angulo >= 2 * M_PI) {
25            angulo -= 2 * M_PI;
26        }
27        vTaskDelay(pdMS_TO_TICKS
28            (50));
29    }
30 }

```

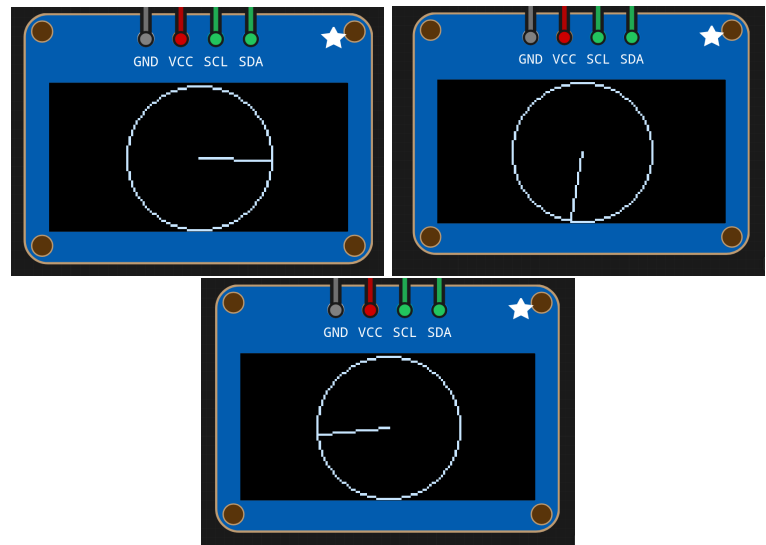


Figure 2: Secuencia completa con todas las figuras.

Conclusión

Es una buena forma de aprender a utilizar el ESP32 debido a que nos ayuda a entender el procedimiento real, a pesar de que el tiempo de simulación con respecto a los retardos puede ser impreciso ya que el simulador depende del procesador de la máquina host de este servicio, en mi caso si docker se sobre carga el tiempo de simulación puede ser afectado. Con respecto al código, la separación de funcionalidad es lo mejor debido a que estamos generalizando el display de figuras sin importar el display que se use, lo único que se cambiaría es el protocolo de comunicación, por otro lado, si se cambia el microcontrolador solo se deben llamar otras librerías para poder hacer el mismo tipo de comunicación I2C dentro de `ssd1306.c` dependiendo del microcontrolador que se use.